COMP 2233: Data Structures

Week 5 Lecture Notes — Pointers, Memory Architecture, Pointer Arithmetic & Dynamic Memory Management

Instructor: Assoc. Prof. Dr. Ömer ÇETİN | Department of Computer Engineering | Academic Year 2026-2027

Table of Contents

[Note: Table of Contents field codes populate automatically upon opening in Microsoft Word or Google Docs.]

1. Fundamentals of Pointers & Physical Memory Architecture

Computer systems organize Random Access Memory (RAM) as a vast, contiguous, zero-indexed sequence of byte-addressable cells. Every variable declared within a program resides in specific memory locations determined by the compiler and operating system runtime. A **Pointer** is a specialized variable whose value is the physical or virtual memory address of another variable or data structure in RAM. Rather than storing direct numeric data values (such as integers or floating-point constants), a pointer stores an address location offset.

> **ARCHITECTURAL ROLE:** Pointers provide direct hardware-level memory access. Understanding pointers is essential for implementing non-contiguous linear structures (Linked Lists), hierarchical structures (Trees), dynamic arrays, and system-level software components where efficiency and memory control are paramount.

1.1 The Address-Of (&) and Dereferencing (*) Operators

C and C++ languages provide two primary unary operators for interacting with memory addresses:

- **Address-Of Operator (&):** Evaluates to the memory address of an L-value operand. For instance, if variable 'x' resides at RAM address 0x7FFE4A00, expression '&x' yields 0x7FFE4A00.
- **Dereferencing / Indirection Operator (*):** Accesses or modifies the value stored at the memory location referenced by a pointer variable. If 'ptr' holds address 0x7FFE4A00, expression '*ptr' accesses the data stored at that address.

```
#include <stdio.h>

int main(void) {
```

```
    int num = 42;
    int *ptr = &num; // ptr stores the address of num

    printf("Value of num: %d\n", num);         // Output: 42
    printf("Address of num (&num): %p\n", (void*)&num); // e.g., 0x7ffd50
    printf("Value stored in ptr: %p\n", (void*)ptr);   // e.g., 0x7ffd50
    printf("Dereferenced *ptr: %d\n", *ptr);    // Output: 42

    *ptr = 99; // Modifies the value of num indirectly
    printf("Updated num after *ptr = 99: %d\n", num); // Output: 99
    return 0;
}
```

1.2 Program Memory Layout: Stack, Heap, Text, and Data Segments

When an executable program runs, the operating system allocates a virtual address space partitioned into distinct memory segments:

Memory Segment	Storage Target & Lifetime	Key Characteristics
Text (Code) Segment	Compiled machine instructions (read-only).	Static size; protected against runtime self-modification.
Data Segments (GData / BSS)	Global and static variables initialized/uninitialized.	Lifetime spans entire execution duration of the process.
Stack Segment	Local variables, function call frames, parameters.	LIFO structure; automatic allocation & deallocation upon function exit.
Heap Segment	Dynamically allocated memory requested at runtime.	Managed manually via malloc/free or new/delete; flexible size.

2. Pointer Categories, Safety Protocols, and Modifiers

In C and C++, uninitialized or improperly managed pointers introduce severe software flaws, including Segmentation Faults (SIGSEGV), illegal memory corruption, and security vulnerabilities. Programmers must adhere to rigorous safety protocols and understand specific pointer taxonomy.

2.1 Null, Wild, Dangling, and Generic Void Pointers

- **Null Pointer (NULL / nullptr):** A pointer explicitly initialized to address 0 (or 'nullptr' in C++11). Indicates that the pointer does not reference any valid memory address. Dereferencing a NULL pointer triggers an immediate crash, making bounds checks mandatory.
- **Wild Pointer:** An uninitialized local pointer variable that contains garbage memory data. Wild pointers must be avoided by initializing pointers upon declaration (e.g., 'int *ptr = NULL;').

- **Dangling Pointer:** A pointer that continues referencing a memory block after that block has been deallocated (freed) or after the target variable goes out of scope. To prevent dangling pointer bugs, set pointers to NULL immediately after freeing memory.
- **Generic Void Pointer (void*):** A generic pointer type that can hold the address of any data type without type binding. Void pointers cannot be dereferenced directly without explicit type casting.

```
#include <stdio.h>
#include <stdlib.h>

void printValue(void *ptr, char type) {
    if (ptr == NULL) return; // Defensive NULL check
    if (type == 'i') {
        printf("Integer: %d\n", *(int*)ptr); // Explicit cast before dereferencing
    } else if (type == 'f') {
        printf("Float: %.2f\n", *(float*)ptr);
    }
}

int main(void) {
    int a = 100;
    float b = 3.14f;

    printValue(&a, 'i');
    printValue(&b, 'f');

    // Preventing Dangling Pointers
    int *dynamic_val = (int*)malloc(sizeof(int));
    *dynamic_val = 500;
    free(dynamic_val);
    dynamic_val = NULL; // Safely neutralize dangling pointer

    return 0;
}
```

2.2 Double Pointers (Pointers to Pointers) & Const Qualification

A **Double Pointer (T**)** is a pointer variable that stores the memory address of another pointer. Double pointers are critical when passing a pointer to a function with the intent of modifying the caller's pointer address (e.g., modifying the head pointer of a Linked List inside a function).

The **const** modifier establishes immutability rules for pointers:

- **Pointer to Constant Data (const int *ptr):** The data value cannot be modified through the pointer ('*ptr = 10' fails), but 'ptr' can be changed to point elsewhere.
- **Constant Pointer (int * const ptr):** The pointer address is fixed upon initialization and cannot point elsewhere ('ptr = &other' fails), but data value can be modified ('*ptr = 10' succeeds).
- **Constant Pointer to Constant Data (const int * const ptr):** Neither pointer address nor target data can be modified.

3. Pointers, Arrays, and Pointer Arithmetic

In C and C++, array names and pointers are intimately linked. An array name 'arr' evaluates to the base address of its first element (&arr[0]). This relationship allows array indexing and pointer arithmetic to be used interchangeably.

3.1 Pointer Arithmetic Mathematical Scaling

When an integer offset 'i' is added to or subtracted from a typed pointer 'ptr', the compiler automatically scales 'i' by the byte size of the target data type (sizeof(T)):

> **POINTER ARITHMETIC FORMULA:** Physical Address = Base Address + (Offset * sizeof(DataType))
> For example, if an 'int' pointer holds address 0x1000 and sizeof(int) = 4, evaluating 'ptr + 2' computes: 0x1000 + (2 * 4) = 0x1008.

```
#include <stdio.h>

int main(void) {
    int arr[4] = {10, 20, 30, 40};
    int *ptr = arr; // Equivalent to ptr = &arr[0]

    printf("arr[0] via ptr: %d (Address: %p)\n", *ptr, (void*)ptr);
    printf("arr[1] via (ptr+1): %d (Address: %p)\n", *(ptr + 1), (void*)(ptr + 1));
    printf("arr[2] via (ptr+2): %d (Address: %p)\n", *(ptr + 2), (void*)(ptr + 2));

    // Pointer Comparison and Offsets
    int *end_ptr = &arr[3];
    printf("Elements between end_ptr and ptr: %ld\n", end_ptr - ptr); // Output: 3

    return 0;
}
```

3.2 Function Parameter Passing: Pass-by-Value vs. Pass-by-Address

By default, C passes arguments by value, creating independent copies inside function call frames. Passing pointers (pass-by-address) enables functions to modify caller variables directly and eliminates the overhead of copying large arrays or structures.

```
#include <stdio.h>

// Pass-by-Address Swap Function
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

```
int main(void) {
    int num1 = 15, num2 = 85;
    printf("Before Swap: num1 = %d, num2 = %d\n", num1, num2);

    swap(&num1, &num2); // Pass addresses
    printf("After Swap:  num1 = %d, num2 = %d\n", num1, num2);

    return 0;
}
```

4. Dynamic Memory Allocation Mechanisms in C & C++

Static and stack memory allocation require sizes to be known at compile time. Dynamic memory allocation allocates RAM blocks from the Heap segment at runtime, allowing applications to handle unpredictable data sizes efficiently.

4.1 C Allocation Functions: malloc, calloc, realloc, and free

Function Syntax	Memory Allocation & Initialization Behavior	Key Use Case
void* malloc(size_t size)	Allocates uninitialized byte block. Contains unpredicted garbage values.	Fast raw memory block allocation.
void* calloc(size_t n, size_t size)	Allocates contiguous memory block and initializes all bytes to zero.	Allocating arrays requiring zero-init.
void* realloc(void* ptr, size_t new_size)	Resizes existing block; moves data to new location if needed.	Dynamic resizing of dynamic buffers.
void free(void* ptr)	Releases heap memory block back to OS allocator pool.	Mandatory memory cleanup; prevents leaks.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 5;
    // Allocate dynamic array of 5 integers initialized to 0
    int *arr = (int*)calloc(n, sizeof(int));
    if (arr == NULL) {
        fprintf(stderr, "Heap Allocation Failed!\n");
        return 1;
    }

    for (int i = 0; i < n; i++) arr[i] = (i + 1) * 10;
```

```
    // Resize array to 8 elements
    int *temp = (int*)realloc(arr, 8 * sizeof(int));
    if (temp == NULL) {
        free(arr);
        return 1;
    }
    arr = temp;

    // Initialize new slots
    arr[5] = 60; arr[6] = 70; arr[7] = 80;

    for (int i = 0; i < 8; i++) printf("%d ", arr[i]);
    printf("\n");

    free(arr); // Clean up memory
    arr = NULL;
    return 0;
}
```

4.2 C++ Operators (new, delete) and Memory Leaks

C++ introduces type-safe operators **new** and **delete**. Unlike 'malloc()', 'new' automatically calculates memory size, returns typed pointers, and invokes class constructors. Conversely, 'delete' invokes destructors before releasing heap blocks. Dynamic arrays require array delete syntax: **delete[] ptr;**

> **MEMORY LEAK HAZARD:** A Memory Leak occurs when heap memory is allocated but never released via free() or delete[]. Over time, unreleased memory accumulates, reducing available RAM and leading to system performance degradation or process crashes.

5. Pointers in Complex Data Structures & Node Allocation

Pointers form the foundational building blocks for non-contiguous data structures, including Singly and Doubly Linked Lists, Binary Search Trees, and Graphs. These structures rely on **Self-Referential Structures** containing data payloads alongside pointers to nodes of the same type.

```
#include <stdio.h>
#include <stdlib.h>

// Self-referential structure node
typedef struct Node {
    int data;
    struct Node *next; // Pointer to next node
} Node;

Node* createNode(int value) {
    Node *newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) return NULL;
```

```
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

int main(void) {
    // Dynamic node chaining
    Node *head = createNode(10);
    head->next = createNode(20);
    head->next->next = createNode(30);

    // Traversal via pointer
    Node *curr = head;
    while (curr != NULL) {
        printf("[Node Data: %d] -> ", curr->data);
        curr = curr->next;
    }
    printf("NULL\n");

    // Clean up heap nodes
    while (head != NULL) {
        Node *temp = head;
        head = head->next;
        free(temp);
    }
    return 0;
}
```

6. Review Questions & Midterm Practice Problem Set

Question 1: Explain the memory layout differences between Stack and Heap allocation. What occurs if a function returns a pointer to a local stack variable?

Question 2: Given 'short arr[5] = {2, 4, 6, 8, 10};' where &arr[0] is 0x2000 and sizeof(short) = 2. What are the hexadecimal addresses and values evaluated by *(arr + 3) and (arr + 3)?

Question 3: Differentiate between 'malloc()' and 'calloc()'. Why is checking for a NULL return value mandatory after dynamic allocation?

Question 4: Write a C function 'void doubleValue(int **ptr_to_ptr)' that dereferences a double pointer to double the integer value of the underlying target variable.